Ce que le Compilateur Java

Accepte ✓ et Refuse ✗

Programmation Orientée Objet — 1ère Année Cycle Ingénieur

Génériques · Collections · Comparable · Comparator · Héritage · Polymorphisme

Ce document recense exhaustivement les cas qui **génèrent une erreur de compilation** et ceux qui sont **acceptés** par le compilateur Java, tels qu'abordés en cours. Chaque règle est illustrée par des exemples concrets avec verdict explicite.

1. Types Bruts (Raw Types) et Génériques

Avant Java 5, les collections étaient non typées (Raw Types). Le compilateur **accepte** le code avec Raw Types mais émet un **avertissement unchecked**. L'erreur réelle survient à l'exécution (ClassCastException).

Code	Verdict du compilateur
`List words = new ArrayList();`	✓ COMPILE — mais warning unchecked (dangereux !)
`words.add("Bonjour"); words.add(new Car());`	✓ COMPILE — le compilateur ne peut pas vérifier
`String s = (String) words.get(1);`	✓ COMPILE — mais ClassCastException à l'EXÉCUTION
`List<String> list = new ArrayList<String>();`	✓ COMPILE — typage sécurisé
`list.add("Bonjour");`	✓ COMPILE — String attendu, String fourni
`list.add(42);`	✗ ERREUR COMPILATION — int ne peut pas aller dans List
`List<int> liste = new ArrayList<int>();`	✗ ERREUR COMPILATION — les primitifs sont interdits dans les génériques

`List<Integer> liste2 = new ArrayList<Integer>();`	✓ COMPILE — Integer (wrapper) est un Object
`List<Integer> li = new ArrayList<>();`	✓ COMPILE — Diamond operator <> : type inféré depuis la déclaration

■ Règle fondamentale — Primitifs interdits

Les types génériques **exigent des objets** (car le Type Erasure travaille avec Object). Il faut impérativement utiliser les classes enveloppes :

int → **Integer** | double → **Double** | char → **Character** | boolean → **Boolean**

L'autoboxing (int ↔ Integer) est automatique au runtime, mais la déclaration doit être correcte.

2. Type Erasure — Ce que le compilateur efface

Après compilation, les paramètres génériques sont **effacés**. La JVM ne voit que des types bruts. Cela crée des cas piégeux :

Code	Verdict du compilateur
`if (list instanceof List<String>)`	✗ ERREUR COMPILATION — impossible de tester le type générique à l'exécution
`if (list instanceof List<?>)`	✓ COMPILE — wildcard accepté pour instanceof
`if (list instanceof List)`	✓ COMPILE — type brut accepté (mais unsafe)
`List<String>.class`	✗ ERREUR COMPILATION — pas de class literal pour type générique
`List.class`	✓ COMPILE — class literal sur type brut
`new T(); // T est un type générique`	✗ ERREUR COMPILATION — impossible d'instancier un type générique directement
`new T[10]; // tableau de type générique`	✗ ERREUR COMPILATION — création de tableau générique interdite

3. Wildcards ? et Bornes (Type Bounds)

Invariance des génériques : List<Integer> n'est PAS un sous-type de List<Number>, même si Integer hérite de Number. Le compilateur **refuse strictement** ce polymorphisme :

Code	Verdict du compilateur
`List<Number> nums = new ArrayList<Integer>();`	✗ ERREUR COMPILATION — invariance : types doivent être identiques
`List<Integer> ints = new ArrayList<Number>();`	✗ ERREUR COMPILATION — même raison
`List<?> any = new ArrayList<Integer>();`	✓ COMPILE — wildcard ? accepte tout type paramétré
`List<? extends Number> nums2 = new ArrayList<Integer>();`	✓ COMPILE — borne supérieure : Integer extends Number
`List<? extends Number> nums3 = new ArrayList<String>();`	✗ ERREUR COMPILATION — String n'étend pas Number

`List<? super Integer> sup = new ArrayList<Number>();`	**✓ COMPILE — borne inférieure : Number est super de Integer**
`public double sum(List<? extends Number> l) {}`	**✓ COMPILE — méthode générique correctement bornée**
`<T extends Comparable<T>> T findMax(List<T> l)`	**✓ COMPILE — T borné : garantit compareTo() disponible**

■ Invariance — Règle à mémoriser

Pour une méthode qui doit accepter List<Integer>, List<Double>, List<Long>... il faut écrire **List<? extends Number>** et non List<Number>.

4. Map et Collections — Erreurs typiques

Code	Verdict du compilateur
`Map<String,Integer> m = new HashMap<String,Integer>();`	✓ COMPILE — déclaration complète correcte
`Map<String,Integer> m2 = new HashMap<>();`	✓ COMPILE — diamond operator
`m.add("Alice", 25);`	✗ ERREUR COMPILATION — Map n'a pas de méthode add() ! Utiliser put()
`boolean b = m.put("Alice", 25);`	✗ ERREUR COMPILATION — put() retourne V (Integer) pas boolean
`Integer old = m.put("Alice", 25);`	✓ COMPILE — put() retourne l'ancienne valeur (ou null)
`for (String k : m) {}`	✗ ERREUR COMPILATION — Map n'est pas Iterable ! Utiliser m.keySet() ou m.entrySet()
`for (String k : m.keySet()) {}`	✓ COMPILE — keySet() retourne un Set itérable
`for (Map.Entry<String,Integer> e : m.entrySet()) {}`	✓ COMPILE — entrySet() : la méthode recommandée
`m.get("Alice") + 1;`	✓ COMPILE — unboxing automatique Integer -> int (attention NullPointerException si absent !)
`TreeMap<String,Integer> tm = new TreeMap<>(); tm.put(null,1);`	✓ COMPILE — mais NullPointerException à l'EXÉCUTION !
`HashMap<String,Integer> hm = new HashMap<>(); hm.put(null,1);`	✓ COMPILE et ✓ EXÉCUTION — HashMap accepte 1 clé null

Méthodes Java 8+ qui compilent / ne compilent pas :

Code	Verdict du compilateur
`m.getOrDefault("Bob", 0);`	✓ COMPILE — Java 8+ : retourne 0 si clé absente
`m.computeIfAbsent("Java", k -> new ArrayList<>());`	✓ COMPILE — Java 8+ : crée la valeur si absente
`m.forEach((k,v) -> System.out.println(k+v));`	✓ COMPILE — Java 8+ : itération avec BiConsumer lambda
`m.entrySet().forEach(e -> e.setValue(e.getValue()+1));`	✓ COMPILE — modification de valeur en cours d'itération via setValue() est OK
`for (String k : m.keySet()) { m.remove(k); }`	✓ COMPILE — mais ConcurrentModificationException à l'EXÉCUTION !

5. Comparable et Comparator — Erreurs de compilation

Pour utiliser **Collections.sort()** ou **TreeSet/TreeMap**, les objets doivent être comparables. Le compilateur vérifie cela strictement :

Code	Verdict du compilateur
`Collections.sort(new ArrayList<Player>());`	✗ ERREUR COMPILATION — Player n'implémente pas Comparable
`class Player implements Comparable<Player> { ... }` `Collections.sort(players);`	✓ COMPILE — Player implémente l'interface requise
`public int compareTo(Player o) { return this.age - o.age; }`	✓ COMPILE — mais DANGER : Integer overflow possible à l'exécution !
`public int compareTo(Player o) { return Integer.compare(this.age, o.age); }`	✓ COMPILE et CORRECT — méthode recommandée
`new TreeSet<Player>(); // Player sans Comparable`	✓ COMPILE — mais ClassCastException à l'EXÉCUTION au premier add()
`new TreeSet<Player>(Comparator.comparing(Player::getAge));`	✓ COMPILE — Comparator fourni, Comparable non requis
`Comparator<Player> c = (p1,p2) -> p1.age - p2.age;`	✓ COMPILE — mais DANGER : Integer overflow si valeurs extrêmes
`Comparator<Player> c2 = Comparator.comparing(Player::getName);`	✓ COMPILE — Java 8+ : méthode de fabrique type-safe
`Comparator<Player> c3 = Comparator.comparingInt(Player::getAge);`	✓ COMPILE — plus efficace : évite l'autoboxing
`players.sort(Comparator.comparing(Player::getName).reversed().thenComparing(Player::getAge));`	✓ COMPILE — chaînage de Comparators (Java 8+)

■ Comparable vs Comparator — Ce que le compilateur vérifie

- **Comparable<T>** : le compilateur vérifie que la méthode **compareTo(T o)** est bien implémentée.
- **Comparator<T>** : le compilateur vérifie que la méthode **compare(T o1, T o2)** retourne un **int**.
  - **Si compareTo() retourne void ou String → erreur de compilation.**
  - **Si compare() oublie le 2ème paramètre → erreur de compilation.**

6. Héritage, Polymorphisme et Cast

Le compilateur vérifie la **compatibilité de type** à la compilation. Certains casts sont détectés comme impossibles, d'autres sont acceptés mais peuvent échouer à l'exécution.

Code	Verdict du compilateur
`Animal a = new Dog();`	✓ COMPILE — polymorphisme : Dog est un Animal (upcasting implicite)
`Dog d = new Animal();`	✗ ERREUR COMPILATION — Animal n'est pas forcément un Dog
`Dog d2 = (Dog) new Animal();`	✓ COMPILE — cast explicite accepté, mais ClassCastException à l'EXÉCUTION
`Dog d3 = (Dog) a; // a contient réellement un Dog`	✓ COMPILE et ✓ EXÉCUTION — downcasting sécurisé car a EST un Dog
`Animal a2 = new Animal(); String s = (String) a2;`	✗ ERREUR COMPILATION — String et Animal sont incompatibles (pas de lien d'héritage)
`if (a instanceof Dog) { Dog d4 = (Dog) a; }`	✓ COMPILE — vérification sécurisée avant cast
`a.bark(); // bark() défini dans Dog, pas dans Animal`	✗ ERREUR COMPILATION — le type déclaré est Animal, bark() inconnu
`((Dog)a).bark();`	✓ COMPILE — cast vers Dog rend bark() accessible

Surcharge (Overloading) et Surcharge (Overriding) :

Code	Verdict du compilateur
`@Override public void speak() { ... } // speak() existe dans Animal`	✓ COMPILE — signature identique, @Override vérifié
`@Override public void speak(String msg) { ... } // speak() sans paramètre dans Animal`	✗ ERREUR COMPILATION — @Override : méthode n'existe pas dans le parent avec ce paramètre
`@Override public String speak() { ... } // speak() retourne void dans Animal`	✗ ERREUR COMPILATION — le type de retour doit être compatible (covariant ou identique)
`@Override private void speak() { ... } // speak() est public dans Animal`	✗ ERREUR COMPILATION — ne peut pas réduire la visibilité lors du override
`@Override protected void speak() { ... } // speak() est public dans Animal`	✗ ERREUR COMPILATION — protected est plus restrictif que public
`@Override public void speak() { super.speak(); }`	✓ COMPILE — appel de la méthode parente via super

7. Interfaces et Classes Abstraites

Code	Verdict du compilateur
`abstract class Shape { abstract double area(); }`	✓ COMPILE — méthode abstraite dans classe abstraite

`Shape s = new Shape();`	**✗ ERREUR COMPILATION — impossible d'instancier une classe abstraite**
`class Circle extends Shape { double area() { return 3.14*r*r; } } Circle c = new Circle();`	**✓ COMPILE — Circle implémente area(), instanciable**
`class Square extends Shape { } // area() non implémentée`	**✗ ERREUR COMPILATION — Square doit implémenter area() ou être abstraite**
`interface Drawable { void draw(); } class Img implements Drawable { }`	**✗ ERREUR COMPILATION — Img doit implémenter draw()**
`interface Serializable { } class Data implements Serializable { }`	**✓ COMPILE — interface marqueur, aucune méthode à implémenter**
`interface A { default void hello() {} } interface B { default void hello() {} } class C implements A, B {}`	**✗ ERREUR COMPILATION — conflit de méthode default, C doit override hello()**
`class C implements A, B { public void hello() { A.super.hello(); } }`	**✓ COMPILE — conflit résolu explicitement**
`interface I { static void util() {} } I.util();`	**✓ COMPILE — méthode statique d'interface appelée via le nom de l'interface**
`interface I2 { int VAL = 10; } // champ`	**✓ COMPILE — les champs d'interface sont implicitement public static final**

8. List, Set, Queue — Erreurs spécifiques

Code	Verdict du compilateur
`List<String> l = new ArrayList<>(); l.get(0); // liste vide`	✓ COMPILE — mais IndexOutOfBoundsException à l'EXÉCUTION
`l.get(-1);`	✓ COMPILE — mais IndexOutOfBoundsException à l'EXÉCUTION
`int i = l.get(0); // List<String>`	✗ ERREUR COMPILATION — String ne peut pas être assigné à int
`String first = l.get(0);`	✓ COMPILE — type correct
`l.add(0, "X"); l.add(5, "Y"); // taille = 1`	✓ COMPILE — mais IndexOutOfBoundsException à l'EXÉCUTION si index > taille
`Set<String> set = new HashSet<>(); set.get(0);`	✗ ERREUR COMPILATION — Set n'a pas de méthode get() !
`set.contains("Alice");`	✓ COMPILE — contains() existe dans Collection
`TreeSet<String> ts = new TreeSet<>(); ts.add(null);`	✓ COMPILE — mais NullPointerException à l'EXÉCUTION
`PriorityQueue<Integer> pq = new PriorityQueue<>(); pq.offer(null);`	✓ COMPILE — mais NullPointerException à l'EXÉCUTION (null interdit)
`Queue<String> q = new Queue<>();`	✗ ERREUR COMPILATION — Queue est une interface, pas une classe !
`Queue<String> q2 = new LinkedList<>();`	✓ COMPILE — LinkedList implémente Queue
`Deque<String> d = new ArrayDeque<>(); d.push("A"); d.pop();`	✓ COMPILE — ArrayDeque : pile LIFO (remplace Stack obsolète)

9. Modificateurs d'accès, static, final

Code	Verdict du compilateur
`private int x; // dans class A new A().x; // depuis class B`	✗ ERREUR COMPILATION — x est privé, inaccessible depuis B
`static void method() { System.out.println(this); }`	✗ ERREUR COMPILATION — this n'existe pas dans un contexte statique
`static void method2() { int x = instanceVar; }`	✗ ERREUR COMPILATION — variable d'instance inaccessible depuis méthode statique
`final int N = 10; N = 20;`	✗ ERREUR COMPILATION — impossible de réassigner une variable final
`final class Parent {} class Child extends Parent {}`	✗ ERREUR COMPILATION — classe final ne peut pas être étendue
`class A { final void display() {} } class B extends A { void display() {} }`	✗ ERREUR COMPILATION — méthode final ne peut pas être overridée

`interface I { public abstract void go(); }`	**✓ COMPILE — les méthodes d'interface sont public abstract par défaut**
`interface I2 { private void helper() { } }`	**✓ COMPILE — Java 9+ : méthodes private autorisées dans les interfaces**

10. Synthèse — Tableau de référence rapide

Règle	✓ Accepté	✗ Refusé
Génériques primitifs	List<Integer>, List<Double>	List<int>, List<double>
Raw Types	Compile avec warning (dangereux)	—
Invariance génériques	List<?>, List<? extends Number>	List<Number> = new ArrayList<Integer>()
instanceof générique	list instanceof List<?>	list instanceof List<String>
Instanciation type générique	Passer une Class<T> en paramètre	new T(), new T[n]
Map.add()	— (méthode inexistante)	map.add("k",v) → Erreur compile
put() retour	V old = map.put(k, v)	boolean b = map.put(k, v)
Itérer Map directement	for (var e : map.entrySet())	for (var e : map) → Erreur compile
TreeMap/TreeSet + null	HashMap → 1 null autorisé	TreeMap/TreeSet → NullPointerException EXÉCUTION
Comparator return	Integer.compare(a, b)	a - b → overflow possible
@Override visibilité	Même ou plus large (public > protected)	Plus restrictive (public → private) → Erreur
Instancier abstraite/interface	Shape s = new Circle() // sous-classe	new Shape(), new Queue() → Erreur compile
Méthode statique + this	this dans méthode d'instance	this dans méthode static → Erreur compile
final reassignment	final int N = 10; (déclaration)	N = 20; après → Erreur compile

■ Les 10 règles à retenir absolument pour l'examen

1. **List<int>** → toujours **List<Integer>**
2. **Map.add()** n'existe pas → utiliser **put()**
3. **put()** retourne **V** (ancienne valeur), pas boolean
4. **for (x : map)** interdit → itérer via **entrySet()**
5. **Set.get()** n'existe pas → utiliser **contains()**
6. **new Queue<>()** interdit → **new LinkedList<>()** ou **new ArrayDeque<>()**
7. **new Shape()** si abstraite → interdit
8. **List<Number> = new ArrayList<Integer>()** → invariance, refusé
9. **this** dans méthode **static** → interdit
10. **TreeMap/TreeSet + null** → compile, plante à l'exécution